



資料結構

Data Structure

Lab 8

姓名： 陳冠豪

學號： 114310125

Lab08-Q1

Modify the inorder traversal function to a postorder traversal function in Part1.

Code

```
#include <iostream> //輸入輸出
#include <queue> //queue 用來做層序
#include <vector> //vector 存資料

using namespace std;

//節點
class TreeNode {
public:
    int value; //存值
    TreeNode* left; //左子
    TreeNode* right; //右子

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} //建構子
};

//二元樹
class BinaryTree {
public:
    TreeNode* root; //建立root

    BinaryTree() : root(nullptr) {} //一開始空樹

    //用陣列建樹(層序)
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr; //沒資料直接回傳

        queue<TreeNode*> q; //queue 存節點
        root = new TreeNode(arr[0]); //第一個當root
        q.push(root); //放進queue

        size_t i = 1; //從第二個開始
```

```

while (!q.empty()) { // 只要 queue 還有
    TreeNode* cur = q.front(); // 拿最前面
    q.pop(); // 移除

    if (i < arr.size()) { // 還有資料就接左
        cur->left = new TreeNode(arr[i]); // 建左節點
        q.push(cur->left); // 放進 queue
        i++; // index 往後
    }

    if (i < arr.size()) { // 還有資料就接右
        cur->right = new TreeNode(arr[i]); // 建立右節點
        q.push(cur->right); // 放進 queue
        i++; // index 往後
    }
}

return root; // 回傳 root
}

// 中序: 左中右
void inorderTraversal(TreeNode* node) {
    if (!node) return; // 空就結束

    inorderTraversal(node->left); // 先左
    cout << node->value << " "; // 印自己
    inorderTraversal(node->right); // 再右
}

// 後序: 左右中
void postorderTraversal(TreeNode* node) {
    if (!node) return; // 空就結束

    postorderTraversal(node->left); // 先左
    postorderTraversal(node->right); // 再右
    cout << node->value << " "; // 最後自己
}
};

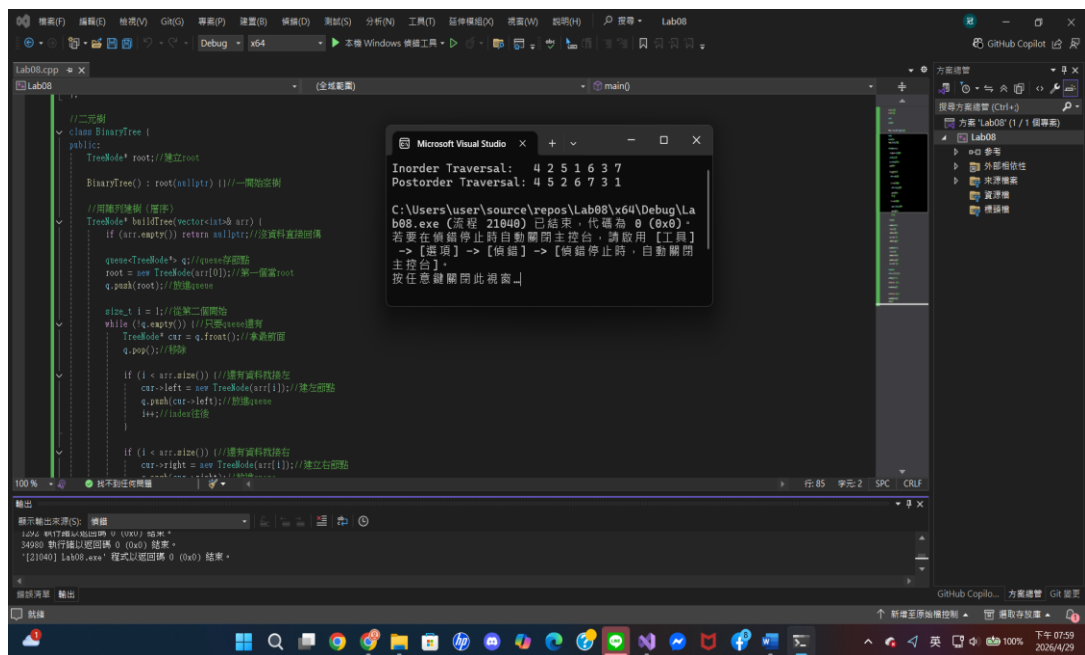
```

```
int main() {  
    BinaryTree tree;//建立 tree  
  
    vector<int> arr = {1, 2, 3, 4, 5, 6, 7};//資料  
    tree.buildTree(arr);//將 arr 丟入 buildtree  
    cout << "Inorder Traversal: ";//提示 Inorder  
    tree.inorderTraversal(tree.root);//中序  
    cout << endl;//換行  
  
    cout << "Postorder Traversal: ";//提示 postorder  
    tree.postorderTraversal(tree.root);//後序  
    cout << endl;//換行  
  
    return 0;//結束  
}
```

Result



```
Microsoft Visual Studio  x  +  v  -  □  X  
  
Inorder Traversal:  4 2 5 1 6 3 7  
Postorder Traversal: 4 5 2 6 7 3 1  
  
C:\Users\user\source\repos\Lab08\x64\Debug\Lab08.exe (流程 21040) 已結束，代碼為 0 (0x0)。  
若要在偵錯停止時自動關閉主控台，請啟用 [工具]  
-> [選項] -> [偵錯] -> [偵錯停止時，自動關閉  
主控台]。  
按任意鍵關閉此視窗...
```



Lab08-Q2

Implement a method to find the maximum value in the left and right subtrees.

Code

```

#include <iostream> // 輸入輸出
#include <queue> // queue 做層序
#include <vector> // vector 存資料

using namespace std;

// 節點
class TreeNode {
public:
    int value; // 存數字
    TreeNode* left; // 左子
    TreeNode* right; // 右子
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 建構子
};

// 二元樹
class BinaryTree {
public:
    TreeNode* root; // 根節點

```

```

BinaryTree() : root(nullptr) {}//一開始是空的

//用陣列建樹(層序)
TreeNode* buildTree(vector<int>& arr) {
    if (arr.empty()) return nullptr;//沒資料直接回傳

    queue<TreeNode*> q;//queue 存節點
    root = new TreeNode(arr[0]);//第一個當 root
    q.push(root);//放進 queue

    size_t i = 1;//從第二個開始
    while (!q.empty()) { //queue 還有就繼續
        TreeNode* current = q.front();//拿最前面
        q.pop();//移掉

        if (i < arr.size()) { //還有就接左
            current->left = new TreeNode(arr[i]);//建左
            q.push(current->left);//放進 queue
            i++; //往下一個
        }
        if (i < arr.size()) { //還有就接右
            current->right = new TreeNode(arr[i]);//建右
            q.push(current->right);//放進 queue
            i++; //往下一個
        }
    }
    return root;//回傳 root
}

//中序:左中右
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return;//空就停
    inorderTraversal(node->left);//先左
    cout << node->value << " "; //印自己
    inorderTraversal(node->right);//再右
}

//找最大值(遞迴)

```

```

int findMax(TreeNode* node) {
    if (node == nullptr) { //如果是空
        return 0; //回傳 0 (假設都是正數)
    }

    int currentMax = node->value; //先當作最大

    int leftMax = findMax(node->left); //去左邊找

    int rightMax = findMax(node->right); //去右邊找

    if (leftMax > currentMax) { //比左邊
        currentMax = leftMax; //更新
    }
    if (rightMax > currentMax) { //比右邊
        currentMax = rightMax; //更新
    }

    return currentMax; //回傳最大
}

};

int main() {
    BinaryTree tree; //建立樹

    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 }; //資料
    tree.buildTree(arr); //建樹

    cout << "Inorder Traversal: "; //提示
    tree.inorderTraversal(tree.root); //中序
    cout << endl; //換行

    if (tree.root != nullptr) { //確保有資料
        int leftSubtreeMax = tree.findMax(tree.root->left); //左子樹最大

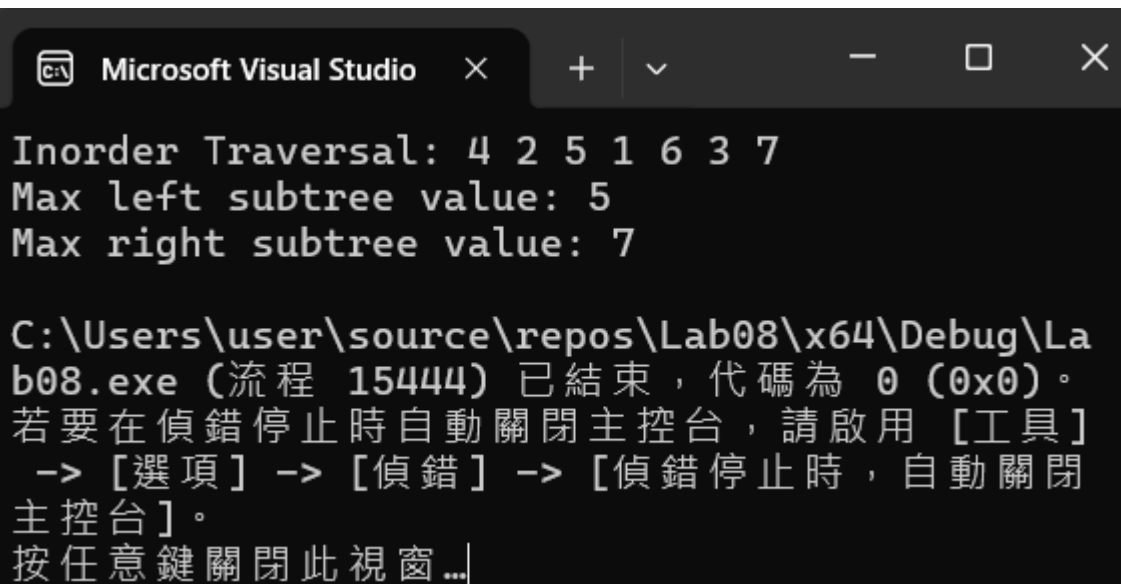
        int rightSubtreeMax = tree.findMax(tree.root->right); //右子樹最大

        cout << "Max left subtree value: " << leftSubtreeMax << endl; //印左
    }
}

```

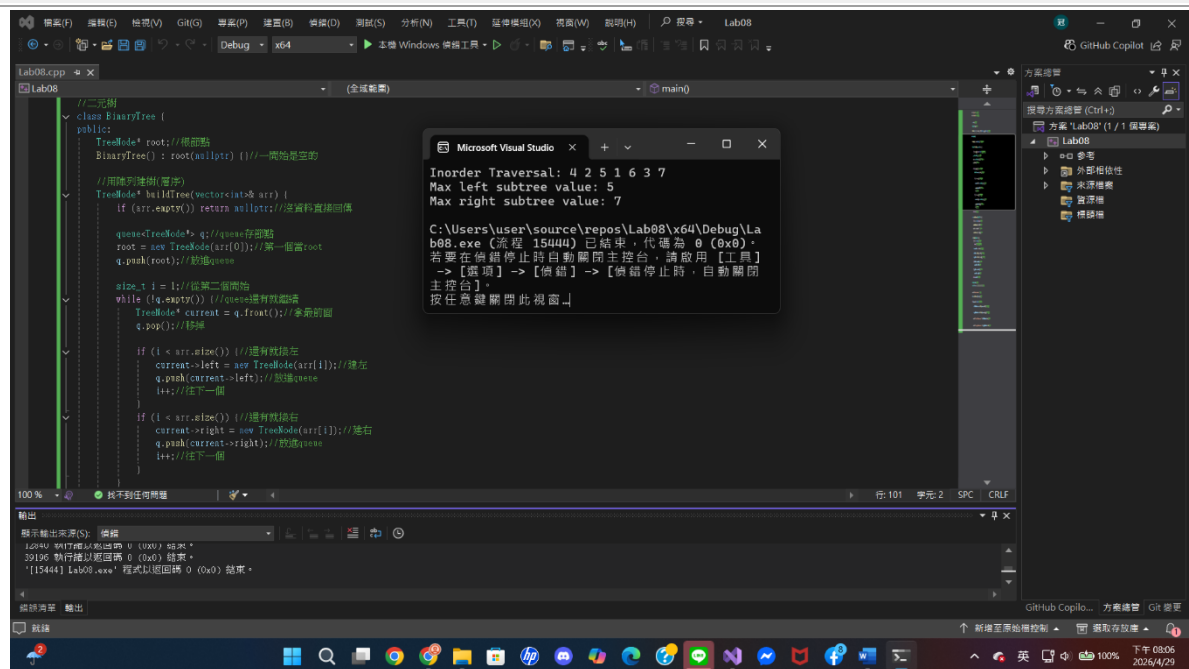
```
    cout << "Max right subtree value: " << rightSubtreeMax << endl; //印右  
}  
  
return 0;  
}
```

Result



The screenshot shows the output window of Microsoft Visual Studio. The title bar reads 'Microsoft Visual Studio'. The output text is as follows:

```
Inorder Traversal: 4 2 5 1 6 3 7  
Max left subtree value: 5  
Max right subtree value: 7  
  
C:\Users\user\source\repos\Lab08\x64\Debug\Lab08.exe (流程 15444) 已結束，代碼為 0 (0x0)。  
若要在偵錯停止時自動關閉主控台，請啟用 [工具]  
-> [選項] -> [偵錯] -> [偵錯停止時，自動關閉  
主控台]。  
按任意鍵關閉此視窗...
```

Discussion

(1).說明你們如何在左右子樹中找到最大值。

利用遞迴的方式去找最大值。做法是從子樹的根節點開始，一路往左跟往右往下找，直到走到底(遇到空節點)為止。在每一個節點，會去比較三個東西：目前這個節點的值、左子樹回傳的最大值、右子樹回傳的最大值，然後把最大的那個留下來。最後再一層一層往上回傳，回到最一開始的地方時，就可以得到整個子樹的最大值。

(2)如果左右子樹不為完整樹，會發生甚麼事情？

就算左右子樹不是完整的(有缺節點)，程式還是可以正常運作，不會出錯。因為在程式裡面有先判斷如果節點是空的(nullptr)，就直接回傳一個預設值(像是 0)，不會再繼續往下找。所以實際上只會對「有存在的節點」做比較，不完整的地方會被自動跳過，因此不會影響最後找出最大值的結果。